

```

static boolean flags = new boolean[2]; // initially zero

public void foo() {
    int i = ThreadId.get(); // F.1
    int j = 1 - i; // F.2
    while (true) { // F.3
        flags[i] = true; // i would like to enter // F.4
        if (flags[j] == false) { // you don't // F.5
            if (flags[i] == true) { // my request was not reset // F.6
                break; // i win // F.7
            }
        } else {
            // looks like we have a contention
            flags[i] = false; // retreat // F.8
            flags[j] = false; // forcibly reset competitor // F.9
        }
    }
}
}

```

Утверждение:

метод foo()

- не wait-free
- не lock-free
- obstruction-free
- не starvation-free
- не deadlock-free

Доказательство: пусть T_0, T_1 потоки с id 0 и 1 соответственно.

Внутри цикла возможны такие последовательности исполнения, что

$$\left[\begin{array}{l} T_0.F.4 \rightarrow T_1.F.4 \\ T_1.F.4 \rightarrow T_0.F.4 \end{array} \right] \rightarrow \left[\begin{array}{l} T_0.F.5 \rightarrow T_1.F.5 \\ T_1.F.5 \rightarrow T_0.F.5 \end{array} \right] \rightarrow T_0.F.8 \rightarrow T_1.F.8$$

Если каждый раз внутри цикла последовательность будет таковой, то цикл никогда не завершится $\Rightarrow$ foo не wait-free и не lock-free.

Докажем, что foo - obstruction-free. Пусть поток T_1 остановлен.

Рассмотрим состояния переменной flags:

- $(\text{true}, \text{false}) \Rightarrow F.4 \Rightarrow F.5 \& F.6 \text{ is } \text{true} \Rightarrow \text{reach } F.7$
- $(\text{false}, \text{false}) \Rightarrow F.9 \Rightarrow F.4 \Rightarrow \text{reduces to } (\text{true}, \text{false})$
- $(\text{true}, \text{true}) \Rightarrow F.4 \Rightarrow F.5 \& F.6 \text{ is } \text{false} \Rightarrow F.8 \Rightarrow F.9 \text{ reduces to } (\text{false}, \text{false})$
- $(\text{false}, \text{true}) \Rightarrow F.8 \Rightarrow F.9 \Rightarrow \text{reduces to } (\text{false}, \text{false})$

для всех случаев цикл завершается за конечное число шагов $\Rightarrow$ foo obstruction-free.

Вернёмся к последовательности исполнения, которая использовалась в качестве доказательства отсутствия wait-freedom и lock-freedom.

Поток T_0 ожидает T_1 , а T_1 ожидает T_0 .

Цикл в wait-for графе $\Rightarrow$ deadlock $\Rightarrow$ foo не deadlock-free $\Rightarrow$ foo не starvation-free.

□